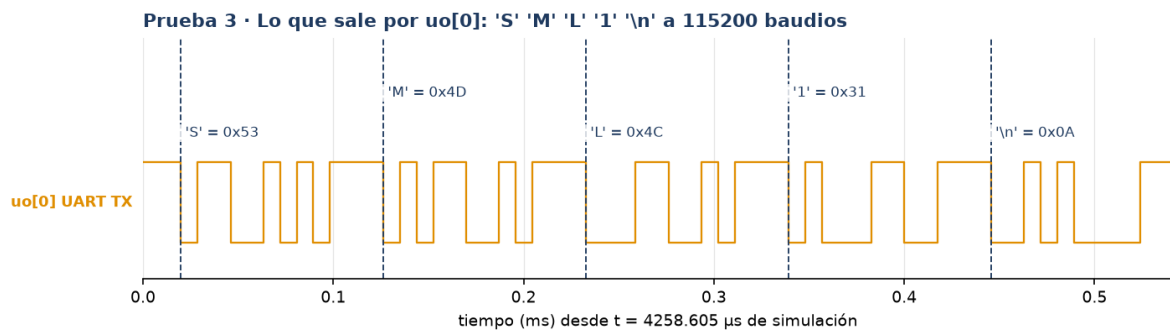


Probando tu chip en la computadora

Testbenches, formas de onda, síntesis con compuertas Sky130 y gate-level del proyecto **SERV TinyML**



Qué probé · cómo lo probé · qué salió · qué NO se pudo probar aquí y por qué

Repositorio github.com/ric1806/tiny, rama `claude/hola-wl31sp` · pruebas ejecutadas el 28 de septiembre de 2026 sobre el commit del diseño `4ae8464`

Contenido

1. Resumen: qué salió	3
2. Cómo lo probé (el montaje)	4
3. Las 10 regresiones originales	5
4. Las 5 pruebas nuevas, una por una	6
5. Gate-level: el chip hecho de compuertas Sky130	11
6. Área, tiempos y lint	13
7. Lo que NO se pudo probar aquí, y por qué	15
8. Cómo repetir estas pruebas tú mismo	15

1. Resumen: qué salió

Probé todo lo que se puede probar sin fabricar el chip y sin las herramientas de diseño físico completas. Esta es la tabla resumen; cada fila tiene su sección más abajo con imágenes.

Prueba	Qué verifica	Resultado
10 regresiones originales del repo	MAC, SPI, puente de memoria, SERV, arranque desde Flash/PSRAM	10/10 PASS
5 pruebas nuevas a nivel de placa (RTL)	El chip completo visto solo por sus pines , con su configuración real (50 MHz, UART 115200, SPI 12,5 MHz)	5/5 PASS
Las mismas 5 en gate-level	Igual, pero sobre el netlist de 8 032 compuertas Sky130 que generé con Yosys	5/5 PASS
Lint (Verilator)	Errores de código que el flujo de Tiny Tapeout rechazaría	0 errores, 5 avisos inofensivos
Síntesis Sky130 (Yosys)	Cuántas compuertas y cuánta área ocupa	65 001 μm^2 , 8 032 celdas
Estimación de tiempos (ABC)	Si el circuito es lo bastante rápido para 50 MHz	~9,0 ns de 20 ns (solo esquina típica)
Utilización por tamaño	En qué número de tiles cabe	3x2 ~72 % · 4x2 ~53 %
Diseño físico completo (LibreLane)	Colocación, ruteo, reloj, hold, DRC, LVS, precheck	NO SE PUDO
Chip real / FPGA	Funcionamiento eléctrico	NO SE PUDO

Hallazgos importantes

- 1 **Corrección a la guía que te entregué:** escribí que `uo[5]` (MAC done) era un pulso de un solo ciclo. **Es falso.** La prueba demostró que ese pin **sube con la primera multiplicación y se queda en 1** hasta que el firmware manda el comando `clear` (sección 4.3). Ya corregí la guía.
- 2 `uo[4:1]` no muestra la clase predicha: muestra los 4 bits bajos de cada resultado del MAC. Al final vale **3**, porque el resultado es `19 = 0001 0011` y solo se ven los 4 bits de la derecha.
- 3 El diseño **sigue funcionando después de convertirlo en compuertas Sky130**, incluida la celda de clock gating `d1clkp_4`.
- 4 En gate-level, `uo[0]` (UART TX) está **indefinido durante los primeros 20 ns** (hasta el primer flanco de reloj). Los CS de memoria no tienen ese problema porque `project.v` los fuerza a 1 durante el reset.
- 5 El camino más lento es una cadena de 32 sumadores (el acumulador del MAC): unos 9 ns en condiciones típicas. En la esquina lenta de fabricación el margen puede reducirse mucho; **hay que confirmarlo con el análisis de tiempos de LibreLane**.
- 6 Con el diseño actual, **3x2 queda en ~72 % de ocupación** (por encima del 70 % recomendado) y **4x2 en ~53 %**.

2. Cómo lo probé (el montaje)

Un **testbench** es un circuito "de mentira" que rodea a tu diseño para probarlo: le da reloj, lo resetea, le conecta memorias simuladas y revisa qué sale por sus pines. Es como poner tu chip en una placa de pruebas, pero dentro de la computadora.

Escribí un testbench nuevo, `test/tb_board.v`, que conecta el **top module real** (`tt_um_quevedo_l_serv_tinym1`) a un modelo de la Flash y la PSRAM a través de sus pines `uio`, exactamente como iría el QSPI Pmod en la demoboard. Las pruebas están en `test/test_board.py` (cocotb, Python).

Pieza	Qué es
Diseño bajo prueba	El top module completo, sin cambios, con sus parámetros de producción: reloj 50 MHz, UART 434 ciclos/bit (115200 baudios), <code>SPI_CLK_DIV = 2</code> (SCK 12,5 MHz).
Memoria simulada	El modelo de Flash + PSRAM que ya existía en el repo (<code>serial_flash_psram_model</code>). Responde a los comandos SPI 0x03 (leer) y 0x02 (escribir) y tiene cargado el firmware <code>serv_alive.hex</code> .
Entradas sin usar	Durante el arranque, las entradas que el diseño no usa (<code>ui_in</code> y <code>uio[7:3]</code> , <code>uio[1:0]</code>) reciben valores aleatorios , para demostrar que no le afectan.
Simulador	Icarus Verilog 12 + cocotb 2.0.1. Las ondas se guardan en <code>tb_board.vcd</code> y las convertí en las imágenes de este documento.
Gate-level	Sinteticé el diseño con Yosys 0.33 y la biblioteca real <code>sky130_fd_sc_hd</code> , y simulé ese netlist con los modelos Verilog oficiales de cada compuerta.

Ojo

El modelo de memoria es simplificado: responde bien a los comandos que usa el chip, pero no simula tiempos eléctricos ni todos los comandos de la W25Q128 o la APS6404L. Sirve para comprobar la lógica, no la parte eléctrica.

```
● ● ● make -f Makefile.board (simulación RTL)

10728765.00ns INFO cocotb.regression
** TEST
*****
** test_board.pins_are_safe_during_reset      PASS      285.00      0.01      46903.27 **
** test_board.first_flash_read_on_the_pins    PASS      5320.00      0.01      370751.80 **
** test_board.boots_and_prints_sml1_on_uart   PASS     4781540.00     10.28     465091.82 **
** test_board.ena_low_freezes_memory_pins     PASS     160280.00      0.38     418640.03 **
** test_board.reset_during_boot_restarts_cleanly PASS    5781340.00     10.72     539400.78 **
*****
** TESTS=5 PASS=5 FAIL=0 SKIP=0               10728765.00     21.41     501130.23 **
*****
```

Captura: salida real de las 5 pruebas nuevas en simulación RTL.

3. Las 10 regresiones originales

Primero volví a ejecutar las pruebas que ya traía el proyecto, desde cero (make clean). Pasaron todas:

```
● ● ● Regresiones originales del repositorio

$ python model/test_int8_reference.py
OK
$ cd test && make (6 regresiones originales del proyecto)
test.test_project PASS
test_mac_mmio.test_mac_mmio PASS
test_mac_mmio.test_mac_against_10000_random_int8_products PASS
test_spi_flash_read.test_spi_flash_word_read PASS
test_spi_mem_bridge.flash_and_psram_reads_are_routed_to_the_right_chip PASS
test_spi_mem_bridge.data_bus_has_priority_over_an_instruction_fetch PASS
test_spi_mem_bridge.psram_word_write_uses_the_spi_program_command PASS
test_serv.test_serv_executes_c_firmware PASS
test_serv_extmem.test_serv_executes_c_firmware_from_serial_memory PASS
test_serv_extmem.test_cpu_clock_is_paused_while_serial_memory_waits PASS
TOTAL: 10 pruebas, 10 PASS, 0 FAIL
```

Captura: las regresiones del repositorio.

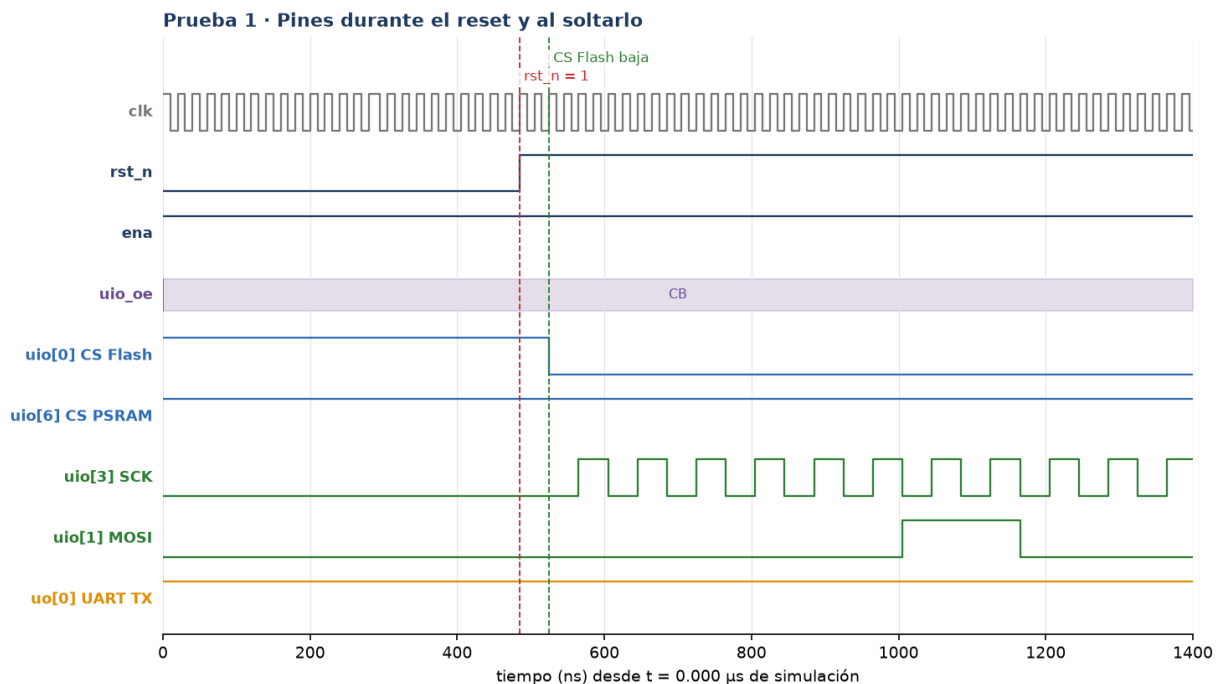
Grupo	Qué comprueba	Resultado
model/test_int8_reference.py	La referencia Python del MAC	OK
Makefile	El top module: pines de memoria en reset y con ena = 0	1/1
Makefile.mac	MAC por MMIO + 10 000 productos aleatorios contra Python	2/2
Makefile.spi	Lectura SPI de una palabra de la Flash	1/1
Makefile.mem	Puente: Flash o PSRAM, prioridad del bus de datos, escritura	3/3
Makefile.serv	SERV ejecuta el firmware C desde memoria ideal	1/1
Makefile.extmem	Arranque desde Flash/PSRAM serie + pausa del reloj	2/2

4. Las 5 pruebas nuevas, una por una

Cada prueba mira el chip **solo por sus pines**, como lo harías con un analizador lógico en el laboratorio. Las imágenes son las formas de onda reales de la simulación: cada fila es un pin, y el tiempo corre hacia la derecha.

4.1 Prueba 1: pines seguros durante el reset

Qué se comprueba: mientras `rst_n = 0`, que la dirección de los pines sea la correcta (`uio_oe = 11001011`), que ninguna memoria esté seleccionada (los tres CS en 1) y que SCK y MOSI estén quietos en 0.

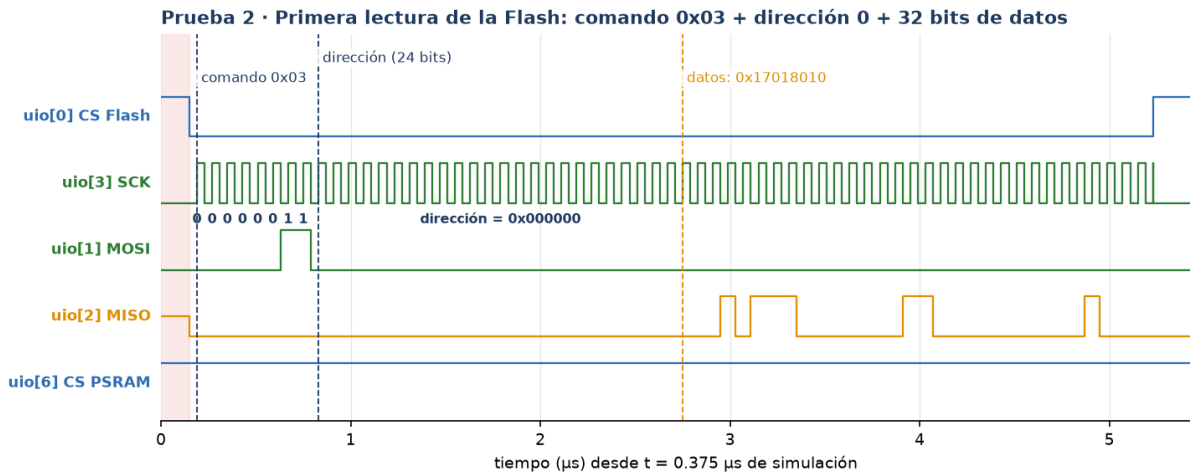


Reset (hasta 485 ns) y arranque. `uio_oe = CB` en hexadecimal = 1100 1011 en binario.

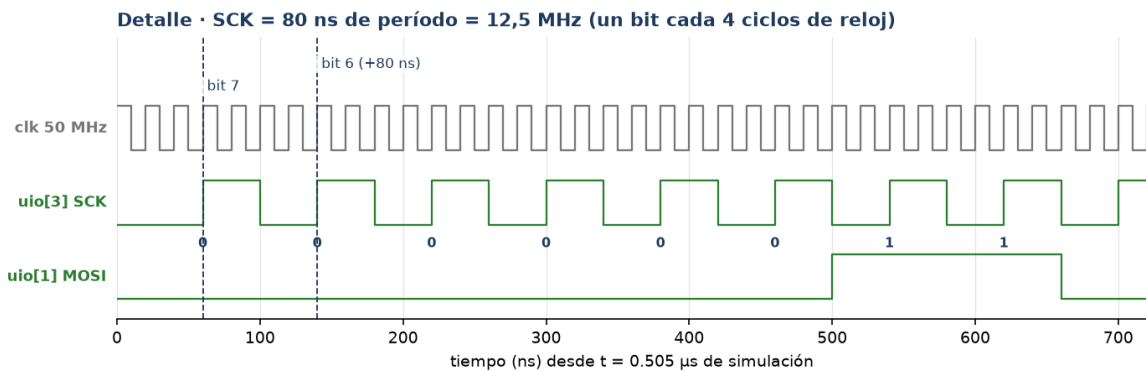
Resultado: PASS. Durante el reset todo está quieto y seguro. 40 ns después de soltar el reset (2 ciclos de reloj), la CS de la Flash baja y empieza la primera lectura. La CS de la PSRAM no se mueve.

4.2 Prueba 2: la primera lectura de la Flash

Qué se comprueba: que lo primero que hace el chip sea pedir la instrucción de la dirección 0 con el comando SPI de lectura (0x03), y que SCK tenga la frecuencia correcta.



Comando por MOSI, respuesta de la memoria por MISO. La franja rosa del principio es MISO sin definir antes de que la memoria sea seleccionada (normal).



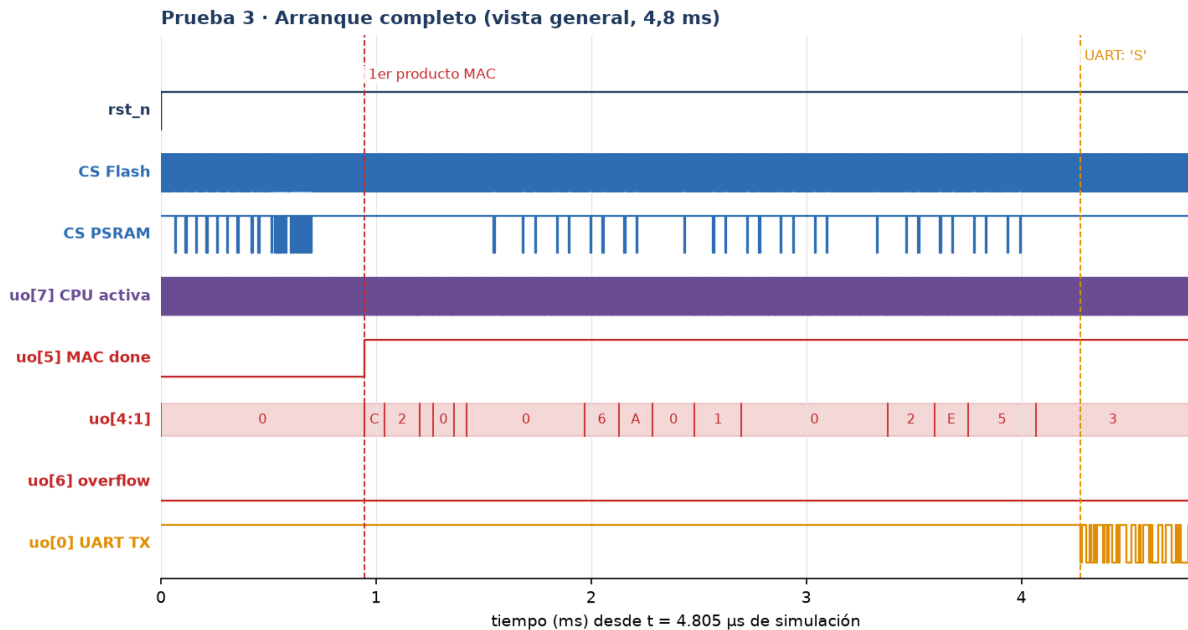
Detalle de los primeros 8 bits: 0000 0011 = 0x03.

Medida	Esperado	Medido en la simulación
Comando + dirección (32 bits por MOSI)	0x03000000	0x03000000
Período de SCK	80 ns (12,5 MHz)	80.0 ns (12.5 MHz)
Palabra devuelta por MISO	1ª instrucción del firmware	0x17018010 = 0x10800117 con los bytes invertidos (little-endian, correcto)
CS de la PSRAM	1 (sin seleccionar)	1

Resultado: PASS.

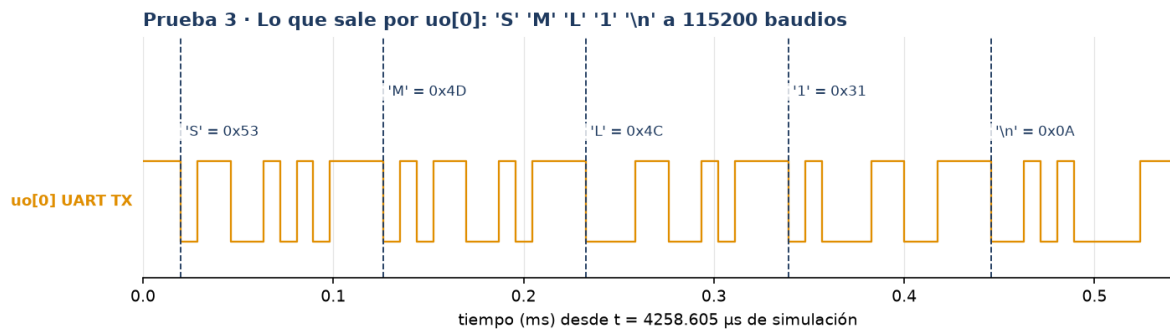
4.3 Prueba 3: arranque completo y mensaje por la UART

Qué se comprueba: el recorrido entero. El chip arranca desde la Flash, copia datos a la PSRAM, ejecuta el programa en C, calcula la red neuronal de prueba con el MAC y envía un mensaje por uo[0]. El testbench se comporta como un **receptor serie** de 115200 baudios y decodifica las letras. Mientras tanto, las entradas sin usar reciben valores aleatorios.

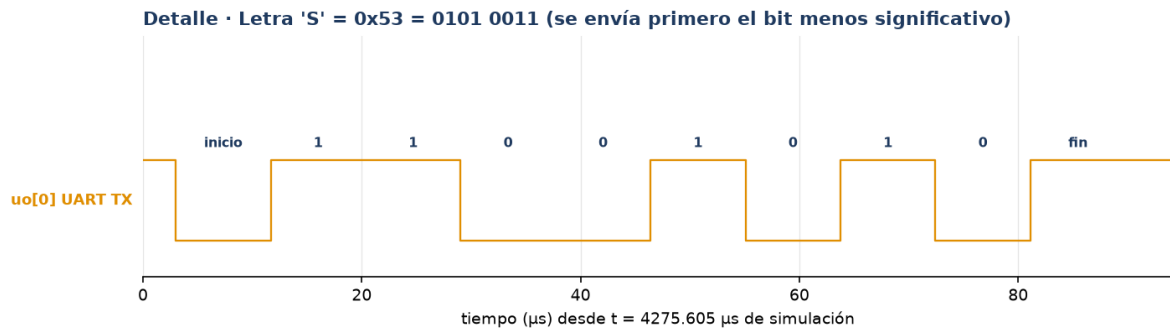


Vista general de los 4,8 ms. CS Flash y CPU activa se ven como bloques porque cambian miles de veces.

Mira la fila `uo[5]` MAC done: sube en 0.95 ms, con la primera multiplicación, y **se queda en 1** hasta el final (1 subida, 0 bajadas en todo el arranque). La fila `uo[4:1]` va mostrando los 4 bits bajos de cada resultado, que cambian con cada operación, y termina en **3**.



Lo que sale por `uo[0]`: cinco caracteres.



Detalle de la letra 'S': bit de inicio, 8 bits de datos (primero el menos significativo) y bit de fin. Cada bit dura 8,68 μs.

Medida	Esperado	Medido
Texto recibido por uo[0]	"SML1\n"	"SML1\n"
Tiempo desde el reset hasta la primera letra	-	4.27 ms
uo[4:1] al final	19 & 0xF = 3	3
uo[5] MAC done al final	-	1 (subió una vez, nunca bajó)
uo[6] overflow	0	0
Efecto de las entradas aleatorias	ninguno	ninguno

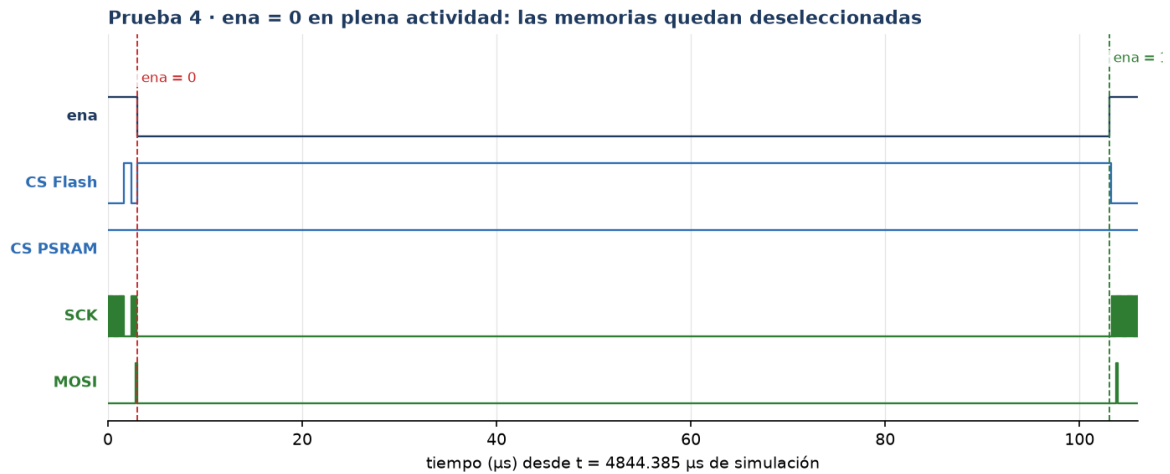
Resultado: PASS.

Corrección importante

En la guía anterior escribí que mac_done dura 20 ns. Es verdad **dentro** del multiplicador (mac_int8.v), pero al pin llega done_q de mac_mmio.v, que es **pegajoso (sticky)**: se pone en 1 al terminar un producto y solo se borra con el comando *clear*. El firmware manda *clear* una sola vez al principio, así que en la práctica queda en 1. Conclusión: sí se ve con un LED, pero **no indica "inferencia terminada"**, solo "ya hubo al menos una multiplicación".

4.4 Prueba 4: ena = 0 congela la memoria

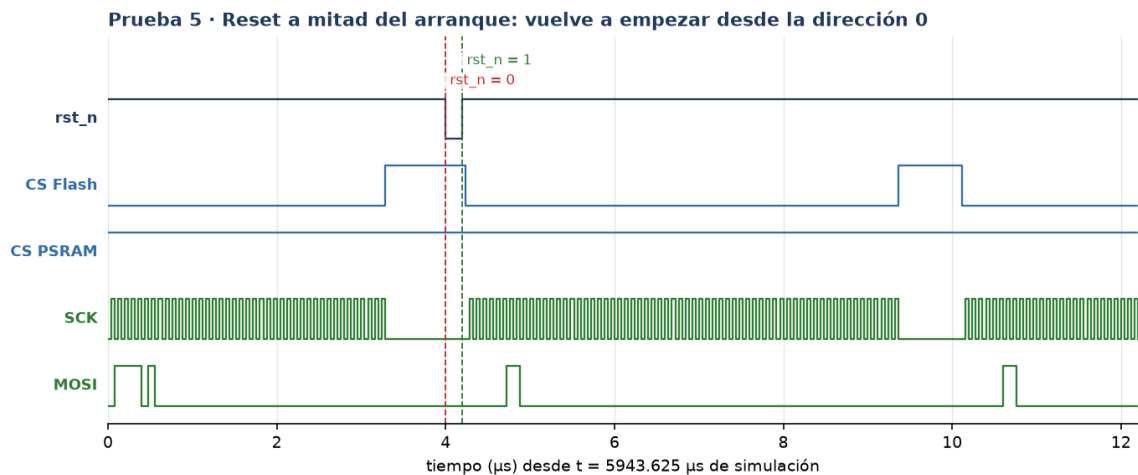
Qué se comprueba: en Tiny Tapeout, ena vale 0 cuando tu diseño no está seleccionado. Si en ese momento el chip siguiera moviendo las memorias, podría molestar. La prueba baja ena en plena lectura y vigila los pines durante 5 000 ciclos (100 μs).



Resultado: PASS. En esos 5 000 ciclos: CS Flash en 0 = 0 veces, CS PSRAM en 0 = 0 veces, SCK en 1 = 0 veces. Todo quieto.

4.5 Prueba 5: reset en mitad del arranque

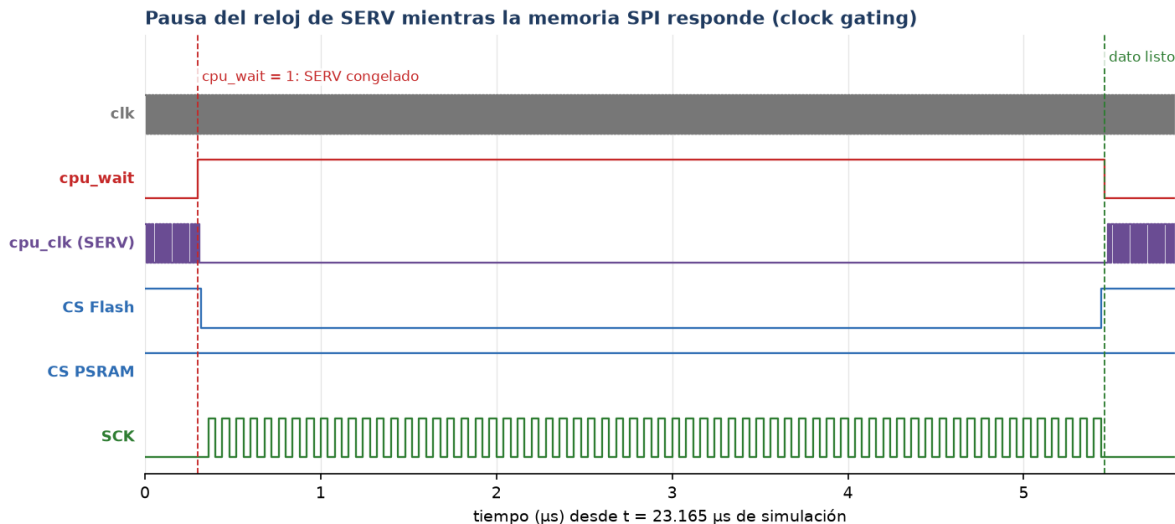
Qué se comprueba: es una tarea pendiente de la Fase 8 del plan ("test de reset durante una inferencia"). Se deja arrancar el chip 1 ms, se aplica un reset y se comprueba que empieza de cero y vuelve a terminar bien.



Resultado: PASS. Tras el reset, el primer comando vuelve a ser 0x03000000 (dirección 0) y el chip vuelve a enviar "SML1n".

4.6 Extra: la pausa del reloj de SERV

Como en RTL se pueden ver señales internas, añadí esta imagen: mientras la memoria responde (cpu_wait = 1), el reloj de SERV (cpu_clk) se detiene por completo, y el SPI sigue trabajando. Cada instrucción que viene de la Flash cuesta unos 5,2 μs.



5. Gate-level: el chip hecho de compuertas Sky130

La simulación RTL prueba el Verilog tal como lo escribiste. La **gate-level** prueba el circuito **después de convertirlo en compuertas reales** de la biblioteca de SkyWater. A veces el Verilog funciona en RTL pero se rompe al sintetizarlo (por ejemplo, con latches o valores sin inicializar).

Lo que hice

- 1 Descargué la biblioteca sky130_fd_sc_hd (el archivo de tiempos .lib y los modelos Verilog de cada compuerta) desde sus repositorios oficiales.
- 2 Sinteticé el top module con **Yosys**, activando SKY130_CLOCK_GATE como hace LibreLane, y excluí las celdas que Tiny Tapeout no permite (familia *lpflow*, *probe*, celdas de retardo y algunas variantes mínimas).
- 3 Simulé el netlist resultante con el **mismo** testbench y las **mismas** 5 pruebas.

```

● ● ● make -f Makefile.board GATES=yes (gate-level, compuertas Sky130)

10728766.00ns INFO cocotb.regression *****
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** test_board.pins_are_safe_during_reset PASS 285.00 0.01 27709.24 **
** test_board.first_flash_read_on_the_pins PASS 5321.00 0.02 272944.97 **
** test_board.boots_and_prints_sml1_on_uart PASS 4781540.00 26.33 181616.20 **
** test_board.ena_low_freezes_memory_pins PASS 160280.00 0.95 167838.55 **
** test_board.reset_during_boot_restarts_cleanly PASS 5781340.00 33.13 174484.04 **
*****
** TESTS=5 PASS=5 FAIL=0 SKIP=0 10728766.00 60.45 177469.73 **
*****

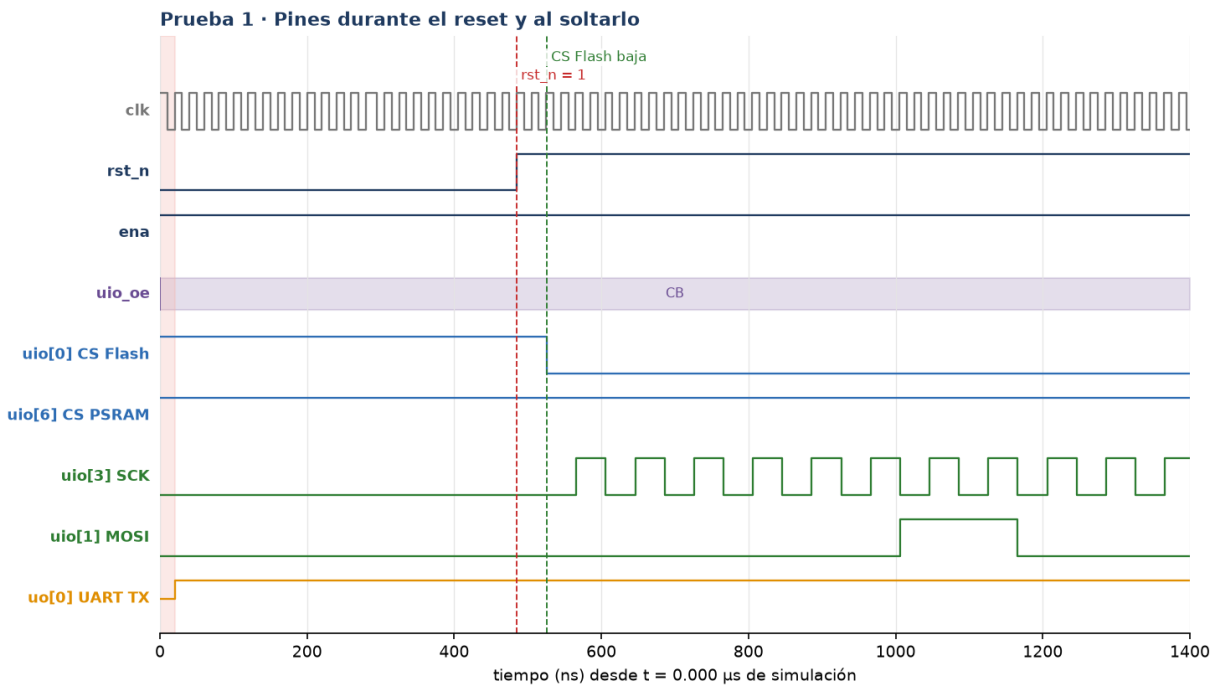
```

Captura: las 5 pruebas sobre el netlist de compuertas.

Medida	RTL	Gate-level
Texto por UART	"SML1\n"	"SML1\n"
Tiempo hasta la primera letra	4.2728 ms	4.2728 ms
Primer comando SPI	0x03000000	0x03000000

Medida	RTL	Gate-level
SCK	12.5 MHz	12.5 MHz
uo[4:1] final	3	3
Reset a mitad de arranque	SML1\n	SML1\n

Resultado: **5/5 PASS**, con los mismos resultados y los mismos tiempos que en RTL.



Gate-level: la franja rosa en uio[0] UART TX son los primeros 20 ns, en los que la salida está indefinida (X).

Detalle encontrado: en el netlist, uio[0] vale X hasta el primer flanco de reloj, porque el reset de la UART es **síncrono** (necesita un flanco de reloj para actuar). En la demoboard el reloj funciona durante el reset, así que es un detalle menor. Si se quisiera eliminar, bastaría con forzar uio[0] = 1 mientras soc_rst_n = 0, igual que ya se hace con los CS.

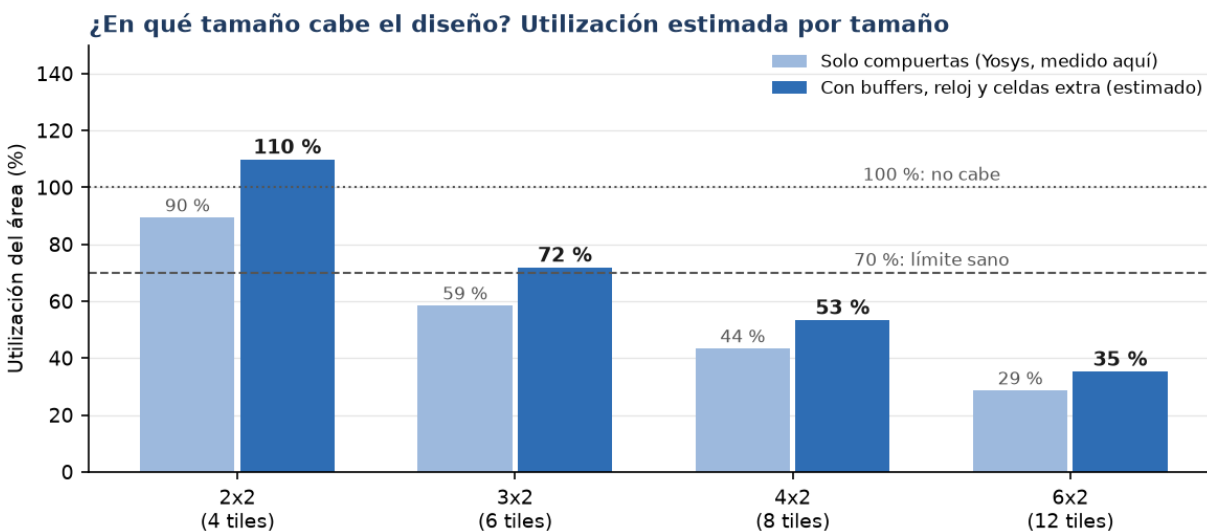
Ojo

Esto **no** es el gate-level oficial de Tiny Tapeout. El oficial usa el netlist que sale de LibreLane **después** de colocar y rutear, que incluye buffers, el árbol de reloj y las celdas de reparación de hold. El mío es el netlist de síntesis (antes de colocar). Aun así, demuestra que la lógica sobrevive a la conversión en compuertas Sky130, incluida la celda sky130_fd_sc_hd__d1clkp_4.

6. Área, tiempos y lint

6.1 Área: cuánto ocupa y en qué tamaño cabe

Yosys cuenta el área de todas las compuertas: **65 001 μm^2** en **8 032 celdas**, de las que 1 609 son flip-flops (el banco de 32 registros de SERV se lleva la mayoría). A eso, el diseño físico le suma buffers, el árbol de reloj, celdas de reparación y celdas obligatorias. Para estimar esa parte usé un dato real del propio proyecto: el hardening en 3x4 dio **35,310 %** de ocupación, lo que equivale a unos **79 750 μm^2** de celdas después del diseño físico.



Tamaño	Área útil (core)	Solo compuertas	Estimado real	Veredicto
2x2	72 565 μm^2	90 %	110 %	No cabe
3x2	110 874 μm^2	59 %	72 %	Al límite
4x2	149 183 μm^2	44 %	53 %	Holgado
6x2	225 802 μm^2	29 %	35 %	Sobra mucho

El área útil de 2x2 (72 565 μm^2) coincide exactamente con la que reporta el proyecto, así que las fórmulas de tamaño son correctas. Los otros tamaños se calculan con el mismo paso de tile. El "estimado real" es una aproximación: el valor definitivo es el que dé LibreLane.

Qué significa para el dimensionamiento

El commit más reciente del proyecto se llama "Repair 3x2 hold timing after routing", o sea: en 3x2 ya se consiguió **rutear** y el problema pendiente era de tiempos de hold. Mi estimación (~72 %) confirma que 3x2 **cabe, pero muy justo**, por encima del 70 % recomendado y sin espacio para lo que falta (QSPI de 4 líneas, registro de clase, UART RX...). **4x2 (~53 %)** deja margen para crecer y para que el hold se repare con más facilidad.

6.2 Tiempos: ¿es lo bastante rápido para 50 MHz?

A 50 MHz cada ciclo dura **20 ns**: toda señal debe ir de un flip-flop al siguiente en menos de eso. Usé ABC (el motor de mapeo de Yosys) con los retardos de las celdas Sky130 para buscar el camino más lento.

Dato	Valor
Camino más lento estimado	9,04 ns (esquina típica: 25 °C, 1,8 V)
Qué es	Una cadena de 32 celdas maj3: el sumador <i>ripple-carry</i> de 32 bits del acumulador del MAC
Margen en la esquina típica	~11 ns, holgado
Lo que falta	Retardo de los cables, desfase del reloj, la esquina lenta (ss, 100 °C, 1,60 V) y todo el análisis de hold

Ojo

En la esquina lenta de fabricación las compuertas pueden ser casi el doble de lentas. 9 ns podrían acercarse peligrosamente a 20 ns. Revisa el reporte de tiempos (STA) de LibreLane en GitHub Actions: debe dar **slack positivo** en todas las esquinas. Si no, se puede bajar la frecuencia (CLOCK_PERIOD) o partir ese sumador.

6.3 Lint con Verilator

Un **linter** revisa el código sin simularlo, buscando errores típicos. LibreLane lo ejecuta con Verilator antes de sintetizar, y si hay errores se detiene.

Modo	Errores	Avisos
Normal (como en el flujo)	0	5
Estricto (-Wall)	0	51 (casi todos señales sin usar dentro de SERV)
Aviso (modo normal)	Archivo	¿Es un problema?
PINMISSING o_ren	cpu/serv/serv_synth_wrapper.v	No. Es código oficial de SERV (no se toca) y la salida no se usa.
WIDTHEXPAND (comparar 16 con 32 bits)	periph/uart_tx.v, mem/spi_mem_ctrl.v	No. Es una comparación con una constante; se puede limpiar por estética.
WIDTHTRUNC (índice de 6 bits en un vector de 32)	mem/spi_mem_ctrl.v (x2)	No. Revisé el código: el contador nunca pasa de 31, así que el índice siempre es válido.

7. Lo que NO se pudo probar aquí, y por qué

Esta sesión corre en un servidor en la nube con acceso a Internet limitado. Esto es lo que quedó fuera:

Qué	Por qué no	Dónde se prueba
Diseño físico completo (LibreLane: floorplan, colocación, CTS, ruteo)	Necesita OpenROAD, Magic, KLayout y Netgen (se instalan con Docker o Nix) y el PDK completo. Las descargas de GitHub Releases y de los servidores del PDK están bloqueadas por la red de esta sesión.	GitHub Actions (gds.yaml), o en tu PC con Docker
Análisis de tiempos real (setup y hold , todas las esquinas)	Requiere OpenSTA y el circuito ya colocado y ruteado. Solo pude estimar el camino combinacional en la esquina típica.	Reporte STA de GitHub Actions
DRC, LVS, antenas, precheck de Tiny Tapeout	Requieren el layout (GDS) y las herramientas anteriores.	Trabajos gds y precheck de GitHub Actions
Gate-level oficial (netlist después del ruteo)	Ese netlist solo lo genera LibreLane.	Trabajo gl_test de GitHub Actions
FPGA	No hay placa FPGA conectada.	Workflow fpga.yaml + una placa ICE40
El chip real, niveles de 3,3 V, el Pmod físico	Necesita hardware.	Laboratorio, cuando llegue el chip

Buena noticia

Lo que sí se probó cubre toda la **lógica**: el diseño hace lo que debe, desde sus pines, tanto en Verilog como convertido a compuertas Sky130. Lo que falta es la parte **física** (dónde va cada compuerta y si llegan a tiempo), que es justamente lo que hacen los robots de GitHub Actions.

8. Cómo repetir estas pruebas tú mismo

Los archivos nuevos están en la carpeta test / del repositorio:

Archivo	Qué es
<code>test/tb_board.v</code>	El testbench: top module + memoria simulada en los pines uio
<code>test/test_board.py</code>	Las 5 pruebas (cocotb)
<code>test/Makefile.board</code>	Para ejecutarlas en RTL o en gate-level

Simulación RTL (tarda unos 25 segundos):

```
cd test
make -f Makefile.board
```

Para ver las ondas:

```
gtkwave tb_board.vcd      # o: surfer tb_board.vcd
```

Gate-level con el netlist que genera GitHub Actions (descarga el *artifact* del trabajo gds, copia el netlist como test/gate_level_netlist.v y pasa un archivo con la lista de modelos de celdas Sky130):

```
make -f Makefile.board GATES=yes GL_DEFINES=-DUSE_POWER_PINS \
  SKY130_HD_MODELS=modelos_sky130.f
```

Si tienes el PDK instalado, el archivo `modelos_sky130.f` solo necesita dos líneas con la ruta completa de tu PDK (cambia `/home/tu_usuario/pdk` por la tuya):

```
/home/tu_usuario/pdk/sky130A/libs.ref/sky130_fd_sc_hd/verilog/primitives.v  
/home/tu_usuario/pdk/sky130A/libs.ref/sky130_fd_sc_hd/verilog/sky130_fd_sc_hd.v
```

`GL_DEFINES=-DUSE_POWER_PINS` hace falta con el netlist de LibreLane, porque trae los pines de alimentación VPWR y VGND.

Qué buscar en la salida

- Al final aparece una tabla: cada prueba debe decir **PASS**.
- El archivo `board_events.json` guarda lo medido: texto de la UART, período de SCK, primer comando, etc.
- Si una prueba falla, cocotb imprime qué comprobación falló y con qué valor.